

CloverLeaf 编译优化挑战

作者: [Charley-xiao](#)



- 00:00

- [序幕](#)
- [第一幕：竞赛概览](#)
 - [简介](#)
 - [竞赛目标](#)
 - [规则](#)
 - [交付须知](#)
- [第二幕：崩落的界域](#)
- [幕间：快速上手](#)
 - [编译](#)
 - [运行](#)
 - [攻克路径推荐](#)
- [第三幕：聚合的塔尖](#)
- [第四幕：回首的天际](#)
- [终幕](#)
- [说明与致谢](#)
- [故障排除](#)
 - [从源码安装 OpenMPI 时, ./configure 报错 working directory cannot be determined](#)
- [附录一：基准时间参考](#)
- [附录二：安装 AOCC 编译器](#)
- [附录三：保姆级教程](#)
 - [使用 Intel oneAPI](#)
 - [使用 GNU 编译器](#)

序幕

漆黑宇宙的尽头，有一条几乎被遗忘的旋臂。那里星光稀薄，像是天幕上被擦去的一笔。就在这片幽暗之中，值夜的观测员艾诺听见了警报——那是一种无法忽视的低沉的嗡鸣，仿佛有人在全宇宙生物的心底敲响巨鼓。他抬头望向主屏，只见一幅灰阶星图中央亮起不祥的方形波纹：

方格海，苏醒了。



旋臂到达临界状态，预计三十分钟后撕裂！

雪崖哨站的百米合金穹顶被强行开启，冷白探照灯刺穿真空，照向远处那片"海洋"。方格海原本安静得像一张铺在星空的暗色丝毯，此刻却闪烁出交错的蓝紫脉冲，一圈圈能量波浪向外翻卷，速度每分钟翻一倍。"按照现在的增幅，二十九分钟后，外旋臂的引力平衡会先瓦解，然后整条臂被撕碎。"首席动力学家柯帕在通讯频道里说。她的声音没有颤抖，但说完那串数字后，舱室陷入短暂的寂静。

应急频道闪烁成一片红海。来自千座文明的代表以全息投影出现在同一张圆桌旁：碳基、硅基，还有被人类称作"流光"的能量体。没有寒暄，没有礼节。"唯一的解决办法是模拟湍流，"柯帕点开一份简报，网格上的涡旋图案像怒吼的巨兽，"可这里不是普通流体，而是二维量子晶格。天枢必须在三十分钟内跑完四个算例，否则——" 她没把结局说出口。所有人都知道那意味着什么。

天枢并非一颗行星，而是一座漂浮在零维奇点上的环形超算。十万台计算塔像灯塔般环抱中央空洞，每台塔身上万条光纤汇成瀑布，顺着金属峭脊奔腾。启动指令下达的瞬间，整座星门亮起比恒星还刺目的白光，照得工程甲板犹如白昼。然而，当输入参数后，调度核心只给出一个答案：

INFO

执行任务所需引擎：CloverLeaf（公元 21 世纪，人类代码）。

那段陈旧程序被尘封在档案最深处，格式各异、注释混乱。它之所以被选择，只因为它用交错网格存储能量与密度，这恰好对应方格海的物理纹理。

编译室位于天枢最深的主机腹地，空气干燥得仿佛随时会碎裂。圆柱形冷却管壁上覆着薄冰，霜气缭绕。工程师、量子算法师、流光译码体并肩而立，眼睛盯着同一块 360° 环幕。屏幕上，古老的 Fortran、C++、OpenMP 指令被高亮标红——兼容性报错如雨。柯帕深吸一口气："从现在开始，我们只有一种敌人：时间。"

DANGER

旋臂预计十五分钟后撕裂！

键盘敲击声如暴雨。每一次回车，编译器便吐出新的错误：寄存器别名冲突、缓存行伪共享、未知指令集.....错误清单像藤蔓疯长，又被一行行修补砍断。汗水沿着护额滴进面颊的照明灯，闪出细小电弧。

"载入算例一——湍流撕裂。"

执行文件启动，数据洪流瞬间灌满主节点。曲线在坐标系里颤抖，像被掐住喉咙的蛇。十秒后，屏幕突然熄灭。

INFO
SYSTEM SHUTDOWN

缓存溢出，溃败。

没人咒骂，也没人退缩。旧错误被标记，替换方案在神经光束里飞速生成。有人接管向量寄存器，有人调低内存对齐阈值，还有人拆掉整段重复计算的内核。

再运行。

这一次，曲线稳住了第一道波峰，却在第二道湍流处崩断。就在计数到 95 % 时，舰桥灯火猛地失压。

DANGER
Terminated with exit code 137

那一瞬，系统骤然撕成乱码；启明号主引擎刚点火便被异常反向推力贯穿，方格海的指数裂缝在星窗外炸开。半个宇宙被卷进 137 号错误，栈回溯化作炽白潮汐，连光都失去指向。在艾诺即将随洪流抹消之际，编译器将他连同残骸压缩成一束单行注释，回溯到最后一次干净提交：几千年前、方格海尚在胎息的年代。

第一幕：竞赛概览

本幕更改日志

2025/6/30：修复了错误的评分计算方式，更改四样例为两样例。

简介

[CloverLeaf](#) 是一个迷你应用程序，它模拟大型流体力学运算中的显式二阶精确方法求解 2D 笛卡尔网格上可压缩欧拉方程的过程。它采用交错网格存储数据，每个单元存储三个值：能量、密度和压力，速度矢量存储在每个单元角。CloverLeaf 由沃里克大学，布里斯托大学，及及一

些其他机构的研究者共同开发，是用于检测超级计算系统扩展性瓶颈和性能可移植性的代表性程序之一。

方程组与数值求解

CloverLeaf 网格采用二维笛卡尔坐标（即将来会扩展到三维）。在每个单元中心存储 **能量、密度、压力** 三个标量，而 **速度向量** 则存储在单元顶点处——这种“部分变量在单元中心、部分在顶点”的布局称为 **交错网格 (staggered grid)**。

可压缩欧拉方程是一组三个偏微分方程，分别描述能量、质量和动量守恒。CloverLeaf 采用 **显式有限体积** 方法给出 **二阶精度** 解，具体流程如下：

1. **拉格朗日步骤**：使用预测—校正 (predictor-corrector) 方案，根据计算出的时间步长 Δt 推进解；此时网格节点随流体速度“漂移”。
2. **对流重映射 (remap)**：为将网格形状恢复到初始位置，采用二阶 Van Leer 算法，对能量、质量、动量在 x/y 两个方向各做一次对流扫描。每一步交替改变扫描先后次序，以保持二阶精度。在重映射时，需要根据流向使用“迎风”侧 (upwind) 数据。虽然网格几何并未真正变形，但通过计算单元面的平均速度来近似各面通量。

欧拉方程属于双曲型方程，解中会出现激波等不连续性，简单的二阶近似会在激波处失效并产生“振铃”现象。为抑制振铃，CloverLeaf 引入 **人工黏性压力**：一旦检测到激波，使局部精度降为一阶，从而保持解的单调性。

时间步长由 **最大声速** 决定—— Δt 不能超过最快声波穿过单元所需时间，再乘以安全系数以确保稳定。此外还做两项检查：防止顶点互相“追尾”，以及避免单元体积变为负值。

为了闭合方程组，需要 **状态方程 (EOS)**。CloverLeaf 采用理想气体 EOS， $\gamma=1.4$ 。目前程序仅支持“单材料”情形，即域内所有状态共享同一物质——未来版本会加入多材料支持。

算法实现

如果只考虑串行架构，算法实现十分直接。但面向现代/未来硬件，CloverLeaf 的设计强调以下方面：

- **内存访问与局部性**
- **编译器优化友好**
- **线程并行 & 向量化**

计算被拆分为大量 **内核 (kernel)**，每个内核只做一件事：按固定模板遍历全网格并更新一组变量。内核内部控制逻辑极少，方便编译器优化。为避免写依赖、提高并行度，必要时会把更新结果写入网格副本。这样每个单元都能独立更新，天然适合线程化和 SIMD 向量化。

边界单元与 Halo 交换

计算域外围通过 **边界条件** 关闭方程。网格周围再包一圈或两圈 **Halo 单元**，为计算模板提供邻域数据，内核本身无需更新这些单元。Halo 数据来源有二：

1. **处理器分区内部**：直接把相邻分区的真实单元值复制过来；
2. **物理边界**：应用物理边界条件（目前仅实现“反射”边界）。

竞赛目标

本次竞赛的目标是对 CloverLeaf 进行编译优化，提升其性能。我们将提供一个 CloverLeaf 的简化版本，包含了两个算例，每个算例都包含了不同的物理模型和参数设置。你需要在给定的时间内完成所有算例的编译优化，并提交你的代码和结果。选手需要使用 GNU 以及 Intel 编译器进行编译优化，而 LLVM/AOCC 为附加 bonus。

参赛队伍需要在 2 台计算节点上完成制定算力的评测，用于计分的指标是问题求解时间，即程序输出的 `clover.out` 最后 1 个 Wall clock.

规则

代码编写规则

- **总体要求**：不能修改计时代码，以及计时代码与其他函数的相对位置。
- **编程语言**：仅限 C、C++ 或 Fortran，兼容版本不低于 C11、C++11 或 Fortran 2008。仅保留一个主入口（C/C++ 的 `main` 或 Fortran 的 `program`）。须使用 MPI-3 与 OpenMP-4 并行。
- **算法要求**：不允许修改物理过程。
- **输入/输出要求**：不得修改输入、输出文件（`*.in`、`*.out`），输出文件的内容与格式须与参考代码完全一致。

测试规则

- **运行环境**：在竞赛指定计算平台运行，规模 ≤ 2 个计算节点、无最多核心要求，程序运行时间 ≤ 30 分钟。
- **测试算例**：共 2 个算例，输入文件位于参考代码 `cases` 目录，不得修改 `clover.in`。

正确性验证

- 组委会以参考 `clover.out` 为基准验证：计算过程中 **Volume** 与 **Mass** 保持不变，最终 **Kinetic Energy** 与参考值偏差 $\leq \pm 0.1\%$ 。

- 对 `case1` 、 `case2` 、 `case3` , 若通过验证, `clover.out` 末尾将输出 "This test is considered PASSED".
- 组委会阅读工程文档和代码, 以确认优化未改变 **CloverLeaf** 的物理过程。
- 若仍无法确认代码正确性, 组委会可要求进一步解释, 并在不同处理器/参数下复测。**未通过正确性验证的算例, 其性能得分计 0 分。**

性能分数

- 性能指标 A_{CL} (单位: 秒) 取自 `clover.out` 最后一行 "Wall clock".
- 设算例 i 的得分为 $M_{CL,i}$, 则 $M_{CL,i} = B \times \frac{A_{CL,i}}{A_{CL,\min}}$.
- 性能总分为各编译器得到的性能分数之和, GNU 及 Intel 的 B 值为 10, 而其他编译器 (作为 bonus) 的 B 值为 4.
- 若 **违反规则、无法复现、篡改输出或恶意利用 Bug**, 性能分数记 0 分。

工程分数

- **工程报告** (PDF) :
 1. **CloverLeaf** 算法流程与代码结构说明
 2. 性能分析、优化与实现过程
 3. 测试结果与分析
 4. 复现方法说明
- **代码可移植性**: 须能在其他服务器编译与验证。
- **创新性**
 - 直接抄袭公开资料 → 0 分
 - 借鉴学术/工业方法并改进 → 1-2 分
 - 提出原创优化方法 → 3-4 分
- **可复现性**
 - 无法复现 → 0 分
 - 可复现提交测试结果 → 1-3 分
 - 步骤清晰、依赖说明完整, 可在多平台 (x86、Arm) 运行 → 2-3 分
- **表达质量**
 - 文字混乱/抄袭 → 0 分
 - 语言通顺、论述清晰 → 1-2 分
 - 语言精练、资料来源准确、图文并茂 → 3 分

交付须知

所有提交都需要遵循以下格式。

- `CloverLeaf_SCC/` 以及各个编译器的运行脚本 `job.<compiler>.slurm`
完整源代码以及作业脚本，供组委会运行验证；附加说明请写在根目录 `README`。
- `CloverLeaf_SCC/results/<compiler>/case<1-2>/*` （赛中可不包含）
每算例需包含：
 - `clover.out`
 - 作业脚本
 - 作业调度系统输出
 - **使用 sbatch**：需提供
 - `clover.out`
 - 1 个 shell 脚本
 - 标准输出 `<JobID>.out`
 - 错误输出 `<JobID>.err`文件名中必须含 **Slurm Job ID**。
- `CloverLeaf_SCC/results/report.pdf` （赛中可不包含）
工程报告（可为 docx 或 pdf）。

第二幕：崩落的界域

剧烈的失重感消散后，艾诺扑通一声落进松软草地。鼻尖传来潮润花香，耳旁是青蛙与虫鸣——这些声音在真空中根本不可能存在。他挣扎着坐起，只见月光下水面微漾，倒映着一块巨石，石上鎏金四字：“南方科技大学”。

他置身地球上南科大校园中央的湖畔。腕表在黑暗里闪烁日期：公元 2025-09-01。

几千年的时差，让他几乎忘了呼吸。

夜色宁静，教学楼旁的路灯投下暖黄光晕。湖对岸，一丹图书馆玻璃幕墙映出星点反射；远处学生宿舍高耸，大部分宿舍的灯尚未熄灭。艾诺跌跌撞撞穿过步道，路过自动贩卖机时才恍然——这里的电子支付还要刷校园卡，量子指纹识别要再等几个世纪。但他无暇感慨。唯有一件事在脑中回荡：**方格海必须被重新封印，而钥匙依旧是 CloverLeaf。**

凌晨两点，理学院大门敞开，灯火通明。一群学生围着陈旧的木桌调试代码，桌面堆满方便面杯。最外侧坐着一位短发女生，聚精会神盯着屏幕。IDE 中赫然是 **CloverLeaf**。

"对不起，打扰一下。"艾诺低声开口，却仍惊起一片侧目。

女生摘下耳机："你也在跑这玩意儿？我叫 hachimi，大四超算竞赛队。"

他们把代码拉到校级超算 "启明 2.0" 上。

第一次编译

OpenMP 版本不匹配

第二次编译

节点掉线

凌晨四点，机房里只剩风扇轰鸣与键盘噼啪。hachimi 揉着通红的眼角："再这样下去，明早比赛都要弃权了。"艾诺心底却在打鼓：未来的大崩落已经提前显形——方格海在他来到 2025 的同一刻，便开始提前躁动。

南科大超算俱乐部交流群（QQ 群号：897073438）像被丢进了炸药：

【曼波】在 04:14:11 说道：

@所有人 有人来帮忙跑下 CloverLeaf 吗？hachimi 有点撑不住了，明天 ddl。/kel/kel/kel

【路人甲】在 04:15:28 说道：

有夜宵吃吗，有就去

【hachimi】在 04:16:33 说道：

来就有，理学院一楼速速

【misaka114514】在 04:19:11 说道：

来咯

不到半小时，理学院一楼人满为患。每一次 `make` 通过，就在门口铜钟上敲一下。钟声回荡在安静的校园，惊醒巡逻的保安，也把更多夜猫子吸引进来。

夜色欲尽，天空泛起鱼肚白。艾诺站在图书馆露台，眺望深圳湾方向的云层。那里，本该宁静的高空等离子层闪过诡异电弧——二维量子网格正在那里成形。他掏出残存的未来量子通信片，折射的全息数据显示：

WARNING

方格海初始裂速：0.03 c，预计 72 小时后进入指数阶段。

时间，比他在 4275 年记录的提前了整整三天。

黎明的钟声划破寂静，喷泉准点点亮。理学院内，最后一次编译结束，屏幕打出绿字：

IMPORTANT

This test is considered PASSED

Wall clock: 17.342 s

铜钟被全场齐拉，声浪滚过湖面，与喷泉水柱交织在晨雾中。然而艾诺清楚，真正的战场并非校赛，而在那片尚未完全显形的方格海。

【艾诺】在 06:48:17 写道：

我们要把这段代码，加固到任何时代、任何硬件都能运行。

"从南科大开始，让时间线改写。"他心想。

湖面晨曦渐亮，映出校园与未来交叠的倒影。崩落的界域已然逼近，而新的抵抗也自此启程。

幕间：快速上手

本幕更改日志

2025/7/2：更改了作业脚本 `job.slurm`，使其不继承环境变量，同时增加了对部分参数的说明。

编译

首先，登录在超算平台的账号，下载[压缩包](#)，使用命令 `tar -xzf CloverLeaf20250622.tar.gz` 解压后进入根目录。

然后，使用 `module load ...` 指令来用指定的编译器来编译 CloverLeaf。你可以使用 `module avail` 来查看平台上可用的编译器及相关库。当然，你也可以选择自己安装喜欢的编译器版本（我们强烈鼓励这种做法！）

最后，可以选择修改 Makefile 中的各个选项，并使用 `make COMPILER=...` 来进行编译。编译成功后，会看到当前目录下有名为 `clover_leaf` 的可执行文件。

运行

在 `CloverLeaf_SCC/` 目录下，新建作业脚本 `job.slurm`，用指令 `sbatch job.slurm` 来提交作业。示例作业脚本如下：

- ▶ `job.slurm`
- ▶ `job.lsf (legacy)`

攻克路径推荐

复现 Baseline

确认 CloverLeaf 在默认编译选项下全部通过校验脚本。

编译选项调参

下面列举一些常用的编译器调参套路，供参考。详细教程见各编译器文档。

步骤	目标	技巧 / 参考
热点剖析	找到热点函数	<code>perf record</code> ， <code>gprof</code> ， <code>-qopt-report</code> (Intel)
编译选项扫描	快速找到合适编译选项	<code>gcc -O3 -march=native -flto</code> ，再叠加 <code>-funroll-loops</code> ， <code>-fprefetch-loop-arrays</code> ；或 <code>icx -ipo -Ofast -xHost</code>
向量宽度	利用 AVX-512 / Zen4 ISA	<code>-qopt-zmm-usage=high</code> (Intel)； <code>-mprefer-vector-width=256</code> (AOCC)
LTO / ThinLTO	跨模块内联、去冗余	<code>-flto=thin</code> (Clang 18)

步骤	目标	技巧 / 参考
PGO	基于 Profiling 调优	<code>-fprofile-generate/use</code> (GCC); <code>-prof-gen/use</code> (Intel)
...	...	...

第三幕：聚合的塔尖

黎明过去不过六个小时，南科大超算中心便被临时改造成一座"灯塔"。启明 2.0 机柜上方的检修平台被拆下护栏，超算队队员们把一块块 LED 调试灯接入总控，总功率足以让整面玻璃幕墙泛出幽蓝光晕。

艾诺站在机房走道尽头，望着这些分布式"萤火"汇成的光柱，恍惚间看见未来天枢环塔的倒影。"塔尖只差最后一块基石，"他低声说，"一块能在任何时代自适应膨胀的编译器。"

午后，hachimi 在 GitHub 私信里收到一份匿名 PR：

Pull Request

Add: temporal-portable backend (TPB) for CloverLeaf

补丁注释出奇简短，却精准修复了先前最棘手的 内存对齐与向量冲突。

更诡异的是，它附带一段时间戳： `4275-09-21T17:03Z` 。

hachimi 告诉了艾诺，艾诺立马就明白了，那是来自天枢末日最后一次成功编译的残影。

"你也许不理解，但是这是未来自己给我们的回信。" 艾诺说。

"要不要用？"hachimi 问道。

艾诺沉默片刻，点头："用。但要先看懂，再接进主干，不能让它变成黑匣子。"

傍晚六点，校园进入用电高峰，供电处发来警告：

WARNING

检测到超额功率请求，请尽快降载。

但塘朗山脚下，铜钟已敲到第九下——最后一次 `make` 开始链接。所有人屏息。风扇像合唱团排山倒海，LED 灯泡则一颗颗熄灭——电流被抽走，投入最终编译。

1 % ... 20 % ... 63 % ... 99 % 进度条在 100 % 停顿半秒，终端打印绿字：

IMPORTANT

```
>> TPB enabled: arch=GENERIC, vec=256-bit  
>> Linking complete.
```

随即，整座启明塔灯火熄灭，瞬息又全部亮起，像夜航的灯塔刺破暮色。

hachimi 立即启动"四大算例"并开启 直播。信号从塘朗山天线跃向高空——

- case 1: 湍流撕裂 → 收敛
- case 2: 质量回喷 → 收敛
- case 3: 共振自稳 → 收敛
- case 4: 临界熵泄漏 → 进度 87 % ... 88 % ...

就在所有人以为结局会与凌晨那次失败如出一辙时，深圳湾方向突然亮起一道银线。那是尚未完全显形的方格海，它的边缘被算例数据牵引，折叠成一道垂直光帘，直指天际。

DANGER

方格海裂速抑制失败！外场指数项再次上升。

"是电力的问题，"艾诺惊呼，"快，把能用上的全部用上！"

机械系和物理系合力调整风冷机架，超频至额定 110 %；其他人递上备用电力变压器，把南科大所有学院的电力投入供电。

在水汽蒸腾的夜空下，塘朗山、启明塔与深圳湾光帘形成三角。



零点零分零秒，全校熄灯。所有剩余电力汇聚到那条 MPI 环路——一根"看不见的电缆"绕过行政楼、宿舍、松禾运动场，最后返回启明塔。

CloverLeaf 第三轮启动，实时输出被投射到主楼外墙：

INFO

Wall clock (global): 12.874 s

STATUS: CONVERGED

与此同时，深圳湾的光帘骤然暗淡，折回一圈圈残光，像潮水退入海床。南科大上空，仅余一簇尾焰般的极光，安静漂移，直至无声消散。

风停，灯复燃。校园里爆发山呼海啸的欢呼，足以把阅湖水面震出涟漪。

hachimi 抱着笔记本冲上顶层露台："Wall clock 12.874 秒！比 baseline 快了近 40 %！"

艾诺却把目光投向更远处——他知道，这只是第一阶段。方格海被推迟，但并未根除；宇宙另一端，真正的天枢仍在倒计时。

"塔尖已聚合，" 他在群里敲下最后一行字，"下一步——连塔成环。"

灯火掩映的夜空下，南科大的剪影像一枚安静的符号，镌刻在时间线的中缝。远处海面，微光起伏，无声预告着下一场波澜。

第四幕：回首的天际

暴风般的呼喊与鼓掌声散去，校园上空只剩细雨。水珠沿启明塔塔尖淌下，折射出淡淡虹光。

艾诺静静伫立于塔顶防雷桅杆旁，任雨丝落在肩头。远处深圳湾已褪去银幕，只留下城市灯火与海雾交融的朦胧轮廓——仿佛什么都不曾发生，又好像一切都被重写。

hachimi 端着两杯冒着热气的豆浆走来，递给他："你该歇一歇了。"

艾诺接过，仰头喝了一口。暖流滑过喉咙，他才意识到自己连续醒着三十七个小时。雨声、心跳、服务器机架尚未彻底平息的风扇轰鸣，此刻交织成一种近乎安宁的节奏。

凌晨 03:12，南科大超算中心管理员在后台日志里发现一条异常留言：

留言板

来自：unknown@chronos.loop

标题：Round #0 complete

正文：

谢谢你们替我们点亮第一座塔。

当你们读到这行字，量子裂隙已锁相，

但真正的环路共有十二座灯塔——

七座在人类文明疆域，

五座远在深空，以光年的尺度彼此守望。

下一段坐标：-12° 15' 58"，+130° 18' 40"

请把火炬传下去。

祝好，

Chronos

管理员一度以为是学生的愚人彩蛋，直到日志显示该留言在 0.74 毫秒内写入又自我抹除，连残影也不剩。

当天午后，南科大小剧场举行了原本属于校赛的颁奖仪式。屏幕上滚动播放着比赛排名，却没人真正把心思放在那里。hachimi 在台上简单致辞后，把奖杯递给艾诺："真正的队长，应该是

你。" 艾诺却把奖杯转交给在场所有参赛者："如果没有每一个人，这座塔就不会亮。"

仪式结束，人群散去。启明塔一层机房里，淡蓝光线依旧闪烁，一如初始。艾诺整理了 README，将仓库设为公开。commit 信息只有一句：

IMPORTANT

pass the torch, to wherever light is needed next

夜幕再次降临，雨已停。艾诺与 hachimi 沿着湖畔下行。湖面如镜，映着塔尖灯火与遥远星群。

"坐标指向太平洋正中央。"hachimi 抬头看星空，"那里连一块礁石都没有。"

"塔未必非要建在陆地。"艾诺笑了笑，"或许是漂浮阵列，或许是海底主机，也可能是另一所学校的灯塔——谁知道呢？"

他们相视而笑，不再多言。脚步声在雨后石阶上回荡，像是另一支无形编译器，正在把新一段时代代码缓缓链接。

终幕

在深圳湾光帘熄灭后的第三天清晨，艾诺再次启动了腕上的量子回溯计。

倒计时的指针依旧停在 00:30:00——与他刚被抛出天枢时一模一样，没有向前，也没有向后。

hachimi 把这枚冰冷的金属圆盘翻来覆去："所以.....你的 '回家' 条件还没满足？"

艾诺点点头，将计时器合上："Chronos 的留言说的是十二座灯塔。南科大只是第一座。要让时空闭环，把方格海彻底钝化，我得和你们一起，把剩下十一座也点亮。"

"那点亮之后呢？"

"倒计时才会继续流动，直到归零的那一刻——我就会被送回 4275 年，去接回真正的终局。"

两人相顾无言。学生宿舍区的晨雾弥散，湖面轻轻荡开一道圆弧，好像在为未来铺路。

hachimi深吸一口气，伸出手："那就别耽搁了，下一站——太平洋坐标 -12°15'58"、+130°18'40"。"

艾诺握住她的手，笑意里有微不可察的释然："等环路闭合，我在 4275 年的天枢，为你们守最后一班岗。"

于是，关于艾诺是否"回得去"的答案，也被写进了那十二座灯塔的行程表——

回去，意味着他们成功拯救了未来；

留在这里，则说明未来已不需要再被拯救。

无论哪一种，都值得他奔赴。

说明与致谢

本赛题为南方科技大学 2025 年超算比赛基础赛道编译优化赛题。本赛题所有资源遵循 [CC BY-NC-SA 4.0](#) 协议，允许非商业性使用与修改。。如有任何问题，请提出 issue 或联系 12211634@mail.sustech.edu.cn. 出题人: [Charley-xiao](#).

故障排除

从源码安装 OpenMPI 时，`./configure` 报错 `working directory cannot be determined`

目前尚不清楚原因。可以手动将所有 `configure` 脚本中的 `working directory` 相关行注释掉。一个可以成功 `./configure` 的版本发布在这个路径下 [/work/share/software/openmpi-5.0.8.tar.bz2](#)，可以使用 `cp` 指令进行复制，并使用指令 `tar -xvjf openmpi-5.0.8.tar.bz2` 进行解压。同时，敬请耐心等待组委会修复。

附录一：基准时间参考

基准设置：2 节点，每节点 24 tasks，每 task 2 threads.

注意

基准时间可能会更新。

算例 \ 参考时间	GNU	Intel	AOCC
case 1	--	375.285696983337 s	--
case 2	--	5.81983780860901 s	--

附录二：安装 AOCC 编译器

AOCC 编译器是 AMD 官方的编译器，支持最新的 Zen5 架构。在本次比赛中，由于集群全部为 Intel 节点而不是 AMD，AOCC 编译器的表现可能不如预期，所以作为 bonus，可选做，其基准分数为 4 分（而不是 10 分）。如果你想使用 AOCC 编译器，可以按照以下步骤安装。

1. 进入 [AMD AOCC 官网](#)，下载最新版本的 AOCC 编译器。

Download with End User License Agreement

For installation guidance, refer to [README](#).

File Name	Version	Size	Launch Date	OS	Bitness	Description
aocc-compiler-5.0.0.tar	5.0	136MB	10/10/2024	Ubuntu®, RHEL®, SLES®, Debian®	32 and 64-bit	This tar file can be untarred and used on the listed distributions. SUSE Linux Enterprise Server does not support compilation of 32-bit applications. It only offers runtime support for 32-bit binaries. For more information, refer to the SUSE documentation . sha256sum: 966fac2d2c759e9de6e969c10ada7a7b306c113f7f1e07ea376829ec86380daa
aocc-compiler-5.0.0_1_amd64.deb	5.0	139MB	10/10/2024	Ubuntu®, Debian®	32 and 64-bit	This .deb file is for Debian based distributions. sha256sum: b937b3f19f59ac901a2c3466a80988e0545d53827900eaa5b3c1ad0cd9dfd0c8
aocc-compiler-5.0.0-1.x86_64.rpm	5.0	140MB	10/10/2024	RHEL®	32 and 64-bit	This .rpm file is for RedHat distribution. sha256sum: f3100d911145672c40033e99df164a37ad4a846e93a5b0a5e602c685af995f62
aocc-compiler-5.0.0.sles15-1.x86_64.rpm	5.0	139MB	10/10/2024	SLES®	64-bit	This .rpm file is for SUSE Linux Enterprise Server distribution. SUSE Linux Enterprise Server does not support compilation of 32-bit applications. It only offers runtime support for 32-bit binaries. For more information, refer to the SUSE documentation . sha256sum: 87708c8dfa6d7d8d217d5bef549db3abb580e8a59b3ce6ecfb912966347b82b1

2. 解压下载的压缩包，并进入解压后的目录。

```
bash
tar -xvf aocc-compiler-5.0.0.tar
cd aocc-compiler-5.0.0
./install.sh
```

3. 这会生成一个 `setenv_AOCC.sh` 脚本，用于设置 AOCC 编译器的环境变量。

4. 运行下面的命令以加载环境变量，然后就可以使用 AOCC 编译器进行编译了。

```
bash
source setenv_AOCC.sh
```

5. 下载 OpenMPI 5.0.8 并解压进入目录。

bash

```
wget https://download.open-mpi.org/release/open-mpi/v5.0/openmpi-5.0.8.tar.gz
tar -xvzf openmpi-5.0.8.tar.gz
cd openmpi-5.0.8
```

6. 启用 AOCC 编译器。

bash

```
./configure CC=clang CXX=clang++ FC=flang \
    --with-wrapper-cc=clang \
    --with-wrapper-cxx=clang++ \
    --with-wrapper-fc=flang \
    --prefix=<你想安装到的目录>/openmpi-aocc
make -j && make install
export PATH=<你想安装到的目录>/openmpi-aocc/bin:$PATH
which mpicc
```

在使用时，执行：

bash

```
export PATH=<你安装到的目录>/openmpi-aocc/bin:$PATH
export LD_LIBRARY_PATH=<你安装到的目录>/openmpi-aocc/lib:$LD_LIBRARY_PATH
```

附录三：保姆级教程

使用 Intel oneAPI

Intel oneAPI 已经在集群上装好，使用以下指令来加载环境变量：

bash

```
source /work/share/intel/oneapi-2023.1.0/setvars.sh
```

此时运行

bash

```
which mpicc
```

应得到

```
/work/share/intel/oneapi-2023.1.0/mpi/2021.9.0/bin/mpicc
```

说明 Intel MPI 的可执行文件已经排在 PATH 前列。

接着进入 CloverLeaf_SCC/ 并编译：

```
make COMPILER=INTEL MPI_COMPILER=mpiifort C_MPI_COMPILER=mpicc
```

bash

然后提交任务：

```
sbatch job.slurm
```

bash

注意：该集群目前提供的 oneAPI 版本较旧（2023.1，对应 Intel MPI 2021.9），因此**只包含经典封装器** `mpicc/mpicpc/mpiifort`，**还不支持** `mpiicx/mpiiicpx/mpiifx`。`mpicc` 与 `mpiicx` 的区别如下：

封装器	默认后端	是否继续维护
<code>mpicc</code>	<code>icc</code> （经典 Intel C 编译器）	仍在 ，但被标记为 <i>classic</i>
<code>mpiicx</code>	<code>icx</code> （LLVM-based Intel C 编译器）	主推

`mpicc` 识别所有 *classic ICC* 特有参数；`-xCORE-AVX512` 等写法对 `mpiicx` 不一定成立。`mpiicx` 支持标准 LLVM/Clang 选项，如 `-march=native`、`-flto=full`，并能与 `-fsanitize`、`-fprofile-generate/use` 等现代工具链联动。

Intel 已宣布 *classic* 编译器将在 oneAPI 2027（暂定）后停止更新安全补丁；届时只剩 `icx/ifx`。因此官方推荐新项目直接用 `mpiicx / mpiifx`，老项目逐步迁移。

对大多数 HPC 内核，两者生成的代码性能非常接近；不过 `icx` 在自动向量化与 OpenMP 5.x 支持上更活跃。若需使用最新 `-qnextgen` 自动并行特性或改进版 OpenMP Offload，务必切到 `mpiicx`。

目前，组委会正在安装更新版本的 oneAPI 以确保大家能用上最新的 `mpiicx / mpiifx`。如果不想等待，你也可以自行安装最新版本。届时，编译 CloverLeaf 的指令将变为：


```
make COMPILER=INTEL MPI_COMPILER=mpiifx C_MPI_COMPILER=mpiicx
```

基准时间也将相应发生改变，敬请关注。

提交任务后，会显示 `Submitted batch job <xxx>`，其中 `<xxx>` 是任务 ID。你可以使用 `squeue` 查看任务状态，或使用 `sacct -j <xxx>` 查看任务详情。

通过 `cat <xxx>.out`，你可以查看任务输出结果。若任务成功完成，输出文件中会显示类似以下内容：

```
=== Correctness Check ===
```

```
Num proc: 48
```

```
? Case 1 : Simulating...
```

```
Clover Version    1.300
```

```
    MPI Version
```

```
    OpenMP Version
```

```
    Task Count    48
```

```
    Thread Count: 2
```

```
Output file clover.out opened. All output will go there.
```

```
✓ Passed (ref=8.0560000000e-02, out=8.0560000000e-02, eps=0.0000%)
```

```
🕒 Wall clock = 375.285696983337s
```

```
? Case 2 : Simulating...
```

```
Clover Version    1.300
```

```
    MPI Version
```

```
    OpenMP Version
```

```
    Task Count    48
```

```
    Thread Count: 2
```

```
Output file clover.out opened. All output will go there.
```

```
✓ Passed (ref=3.0750000000e-01, out=3.0750000000e-01, eps=0.0000%)
```

```
🕒 Wall clock = 5.81983780860901s
```

```
=== Summary ===
```

```
Passed: 2, Failed: 0
```

```
Wall clock per case:
```

```
Case 1   : 375.285696983337 s
Case 2   : 5.81983780860901 s
All cases passed within 0.5% tolerance.
```

如果最后显示 `All cases passed within 0.5% tolerance.` , 则表示你的代码通过了所有测试用例。那么恭喜你, 接下来的任务就是做性能分析以及优化了! 祝你好运!

使用 GNU 编译器

这里给出一种自己编译安装的通用做法, 既不会用到系统自带 GCC 8.5, 也能确保 OpenMPI 5.0.8 完全用 GCC 15.1 编译。

1. 编译并安装 GCC 15.1

```
bash
wget https://ftp.gnu.org/gnu/gcc/gcc-15.1.0/gcc-15.1.0.tar.xz
tar -xf gcc-15.1.0.tar.xz
mkdir gcc-15.1.0/build && cd gcc-15.1.0/build

../configure --prefix=<你想安装到的位置>/gcc/15.1 \
              --enable-languages=c,c++,fortran \
              --disable-multilib

make -j$(nproc)
make install
```

2. 激活 GCC 15.1

```
bash
export PATH=<你安装到的位置>/gcc/15.1/bin:$PATH
export LD_LIBRARY_PATH=<你安装到的位置>/gcc/15.1/lib64:$LD_LIBRARY_PATH
```

IMPORTANT

每次使用 GCC 15.1 前, 都需要运行上述两条命令来激活它。

3. 用 GCC 15.1 编译 OpenMPI 5.0.8

```
bash
wget https://download.open-mpi.org/release/open-mpi/v5.0/openmpi-5.0.8.tar.gz
tar -xzf openmpi-5.0.8.tar.gz
mkdir openmpi-5.0.8/build && cd openmpi-5.0.8/build
```

```
../configure --prefix=<你想安装到的位置>/openmpi/5.0.8-gcc15 \  
CC=<你安装GCC的位置>/gcc/15.1/bin/gcc \  
CXX=<你安装GCC的位置>/gcc/15.1/bin/g++ \  
FC=<你安装GCC的位置>/gcc/15.1/bin/gfortran
```

```
make -j$(nproc)  
make install
```

4. 激活 OpenMPI 5.0.8

```
export PATH=<你安装到的位置>/openmpi/5.0.8-gcc15/bin:$PATH  
export LD_LIBRARY_PATH=<你安装到的位置>/openmpi/5.0.8-gcc15/lib:$LD_LIBRARY_PATH
```

bash

IMPORTANT

每次使用 OpenMPI 5.0.8 前，都需要运行上述两条命令来激活它。

5. 验证

```
mpicc --version # 应显示 "gcc version 15.1.0 ..."  
mpirun --version # 应显示 "Open MPI 5.0.8"
```

bash

接着进入 `CloverLeaf_SCC/` 并编译：

```
make COMPILER=GNU
```

bash

新建任务脚本 `job.gnu.slurm`，内容如下：

► `job.gnu.slurm`

然后提交任务：

```
sbatch job.gnu.slurm
```

bash

附录四：提交评测前检查

请确保你的仓库结构如下或者至少包含如下结构：

```
CloverLeaf_SCC/
├─ cases
│   └─ case1
│       └─ clover.in
│       └─ clover.out
│   └─ case2
│       └─ clover.in
│       └─ clover.out
├─ gnu
│   └─ job.gnu.slurm
│   └─ clover_leaf （注意：此处应为 GNU 编译器生成的可执行文件）
├─ intel
│   └─ job.intel.slurm
│   └─ clover_leaf （注意：此处应为 Intel 编译器生成的可执行文件）
├─ aocc （可选）
│   └─ job.aocc.slurm
│   └─ clover_leaf （注意：此处应为 AOCC 编译器生成的可执行文件）
└─ README.md （仓库结构清楚的情况下，在赛中评测可不包含）
```



点亮命运的灯塔

Previous page

[Rust LWE 编程挑战](#)

Next page

[GEMM编程挑战](#)